

ATIVIDADE 02 · RELATÓRIO TÉCNICO

ReUse com voz: assistente virtual com IBM Watson

Entrega técnica completa para automatizar tarefas reais e orientar usuários com Actions determinísticas, Web Chat e uma extensão OpenAPI segura.

50% + 50%

Automação e orientação

Três operações reais sobre anúncios da conta conectada e seis fluxos de ajuda em português.

Actions

Web Chat

OpenAPI 3.0

Next.js

Prisma

Pull request #3

github.com/lucasbuzato/ReUse/pull/3

Código da atividade

github.com/lucasbuzato/ReUse/tree/feat/watson-assistant-voice

Aplicação pública

reuse-lucasbuzatos-projects.vercel.app

Status em 24/09/2026

Código, testes, documentação e artefatos IBM estão prontos. Login/2FA, importação na conta IBM, Preview, Publish e os três IDs do Web Chat permanecem como etapas externas.

Dois blocos, uma experiência coerente

A baseline prioriza short-tail determinístico: frases frequentes geram respostas previsíveis, auditáveis e seguras. IA generativa permanece opcional.

50%

Realização de tarefas

Automação conectada ao ReUse

- Consultar o resumo dos anúncios da conta.
- Pausar todos os anúncios DISPONÍVEL .
- Reativar todos os anúncios PAUSADO .
- Pedir confirmação antes de qualquer mudança em lote.
- Responder de forma idempotente quando não há item elegível.

50%

Orientação ao usuário

Ajuda determinística em português

- Anunciar um item.
- Encontrar itens e usar filtros.
- Demonstrar interesse com segurança.
- Gerenciar anúncios e interessados.
- Entender os quatro status.
- Privacidade e boas práticas.

Decisão de produto

O assistente não inventa respostas para operações críticas. As Actions conduzem condições, confirmação, chamada da extensão e resposta final.

Decisão técnica

O navegador fornece contexto de sessão, mas a API deriva a identidade somente de um token assinado pelo servidor. Nenhum `userId` arbitrário é aceito.

short-tail determinístico -> Actions + condições + extensão
long-tail opcional -> watsonx.ai/Granite como evolução futura
requisito da entrega -> não depende de IA generativa

Tarefas executadas no ReUse

1

Consultar meus anúncios

Retorna total, contagem por status e listas resumidas dos itens disponíveis e pausados.

GET /api/assistant/actions/items/summary

2

Pausar anúncios disponíveis

Após confirmação, altera apenas itens **DISPONIVEL** do proprietário autenticado para **PAUSADO**.

POST /api/assistant/actions/items/pause

3

Reativar anúncios pausados

Após confirmação, altera apenas itens **PAUSADO** do proprietário autenticado para **DISPONIVEL**.

POST /api/assistant/actions/items/reactivate

Fluxo conversacional das mutações

Identificar intenção

frases de exemplo



Consultar resumo

quantidade elegível



Confirmar

nenhuma mudança antes do "sim"

Chamar extensão

token + API key



Atualizar Prisma

ownerId + status



Responder

affectedCount + mensagem

Idempotência

Repetir uma pausa ou reativação sem itens elegíveis retorna sucesso com zero alterações, não um erro.

O chat não escolhe o proprietário

A identidade nasce do cookie HTTP-only já usado pelo ReUse e chega à extensão em um token HMAC curto. O banco sempre filtra por `ownerId` derivado desse token.



Dúvidas frequentes cobertas

1. Como anunciar?

Entrar, selecionar Anunciar, preencher título, descrição, conservação e categoria, adicionar imagem opcional e publicar.

2. Como encontrar itens?

Abrir Explorar, buscar por nome, descrição ou cidade e combinar o termo com filtros de categoria.

3. Como demonstrar interesse?

Entrar, abrir um item disponível e enviar uma mensagem de 10 a 500 caracteres. Um interesse por pessoa e item.

4. Como gerenciar anúncios?

Abrir Perfil para pausar, reativar, reservar, concluir como doado, excluir e acompanhar interessados.

5. O que significam os status?

Disponível aceita interesses; Pausado fica oculto; Reservado está em negociação; Doado está concluído.

6. Como proteger meus dados?

Nunca enviar senha, documentos ou dados bancários; combinar entrega em local seguro e compartilhar apenas o necessário.

Exemplos reconhecidos

“Quero pausar minhas ofertas ativas.”

“Como cadastro um novo item?”

“Onde vejo as pessoas interessadas?”

“Reative meus anúncios.”

“Como busco por cidade?”

“Um item pausado aparece no catálogo?”

Coerência com a aplicação

As respostas refletem rotas, campos, limites e regras reais do ReUse. O assistente não descreve telas ou capacidades inexistentes.

Artefatos prontos para configurar

Artefato	Uso	Caminho
OpenAPI 3.0 JSON	Importar a extensão customizada	<code>watson/reuse-assistant-extension.openapi.json</code>
Catálogo das Actions	Copiar frases, passos, condições e respostas	<code>watson/actions/action-catalog.json</code>
Guia operacional	Executar configuração, Preview e Publish	<code>docs/IBM-WATSON-SETUP.md</code>
Loader Web Chat	Incorporar widget e contexto de sessão	<code>components/WatsonAssistantChat.tsx</code>

Passos restantes na conta IBM

01 · Acessar

Login, 2FA, termos, região e instância watsonx Assistant.

02 · Importar

Carregar o OpenAPI e confirmar as três operações.

03 · Autenticar

Cadastrar a API key da extensão sem expô-la.

04 · Montar

Criar as oito Actions a partir do catálogo versionado.

05 · Validar

Executar Preview e Inspector, ocultando tokens nas imagens.

06 · Publicar

Promover para Live e copiar os três IDs públicos do Embed.

Segredos nunca entram no GitHub ou neste PDF.

`ASSISTANT_ACTION_SECRET` fica apenas no servidor. `ASSISTANT_EXTENSION_API_KEY` fica na Vercel e na configuração privada da extensão.

Resultados técnicos locais

16

testes descobertos

15aprovados
localmente**0**

falhas

4rotas do Assistant
no build

Validação	Resultado observado
<code>npm test</code>	15 aprovados; integração PostgreSQL pulada sem banco local.
<code>npm run lint</code>	Código de saída 0.
<code>npm run typecheck</code>	Código de saída 0.
<code>next build --webpack</code>	Código 0; TypeScript, páginas e Route Handlers reconhecidos.
Sessão anônima	HTTP 401 e resposta privada sem cache.
Chave ausente/incorreta	HTTP 401, sem alcançar o banco.
Token sem escopo de pausa	HTTP 403, sem alcançar o banco.
Playwright em <code>/login</code>	HTTP 200, zero erros, sem overflow e nenhum script IBM sem IDs.
Varredura de segredos	34 arquivos staged; zero assinaturas conhecidas.

CI preparada

PostgreSQL 16 efêmero, migrations,
`RUN_DATABASE_TESTS=1` e quality gate completo.

Limite local documentado

Turbopack rejeita o symlink externo de
`node_modules` no MCP. O checkout normal da CI é
a prova definitiva.

Implementação, evidência e próximo aceite

Arquivos-chave

Token e autorização

```
lib/assistant-action-token.ts
```

```
lib/assistant-request-auth.ts
```

Automação e isolamento

```
lib/assistant-item-actions.ts
```

```
lib/prisma-assistant-item-store.ts
```

APIs e Web Chat

```
app/api/assistant/**
```

```
components/WatsonAssistantChat.tsx
```

Artefatos e testes

```
watson/**
```

```
tests/*assistant*.test.ts
```

Critérios de pronto

- ✓ APIs, tokens, escopos e ownerId.
- ✓ Estado PAUSADO na API e interface.
- ✓ OpenAPI, Actions e guia IBM.
- ✓ Testes, build local e smoke HTTP/UI.
- ✓ Branch e PR separados.
- CI com PostgreSQL e Turbopack.
- Preview, Inspector e Publish IBM.
- Widget real e evidências finais.

Links verificáveis

Repositório

github.com/lucasbuzato/ReUse

Pull request #3

[Entrega da Atividade 02](#)

Branch

[feat/watson-assistant-voice](#)

Aplicação

[Produção na Vercel](#)

IBM · Extensão

[Criar extensão](#)
[Chamar e inspecionar](#)

IBM · Web Chat

[Incorporar no site](#)

Próximo aceite

Concluir a configuração na conta IBM, aguardar a CI verde, validar o widget em produção e substituir os itens pendentes por evidências datadas e screenshots higienizados.